Measuring latency

Load Testing Overview

- Load testing measures service behavior under expected traffic.
- Key metrics:
  - **Latency** (response time)
  - **Throughput** (requests per second)
  - **Error rate**
- Tools by protocol:
  - REST: `hey`, `Locust`, `Vegeta`
  - gRPC: `ghz`, `grpcurl`
  - GraphQL: `Locust`

---

hey – REST Load Testing

- Lightweight CLI for HTTP load testing.
- Installation (Go):

```
go install github.com/rakyll/hey@latest
```

- Usage example:

```
hey -n 1000 -c 50 http://localhost:8080/products
```

- `-n 1000`: total requests
- `-c 50`: total threads

Vegeta – REST Load Testing

- Flexible HTTP load testing tool for REST or any HTTP service.
- Can be used as CLI or library (Go).
- Installation (Go):

```
go install github.com/tsenart/vegeta@latest
```

- Usage example – **attack mode**:

```
echo "GET http://localhost:8080/products" | vegeta attack -duration=30s -rate=50 | vegeta report
```

- `-duration=30s`: total test duration
- `-rate=50`: requests per second
- Usage example – **generate report**:

```
echo "GET http://localhost:8080/products" | vegeta attack -duration=30s -rate=50 | vegeta report -type=text
```

- Can produce reports in **text, JSON, or plots**:

```
vegeta plot < results.bin > plot.html
```

- Can read a file of requests:

```
vegeta attack -targets=targets.txt -rate=100 -duration=1m | vegeta report
```

- Good for **scriptable, high-throughput HTTP testing**.

ghz – gRPC Load Testing

- CLI and Go library for gRPC services.
- Installation (Go):

```
go install github.com/bojand/ghz/cmd/ghz@latest
```

- Usage example:

```
ghz --proto product.proto \
    --call ProductService.GetAllProducts \
    --in '{}' \
    -n 1000 \
    -c 50 \
    localhost:9090
```

---

grpcurl – gRPC CLI

- Like `curl` for gRPC.
- Installation (Go):

```
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest
```

- Usage example:

```
grpcurl -plaintext localhost:9090 list
grpcurl -plaintext localhost:9090 describe ProductService
grpcurl -plaintext -d '{}' localhost:9090 ProductService.GetAllProducts
```

Locust – Customizable Load Testing

- Python-based, supports REST, GraphQL, WebSockets.
- Installation:

```
python3 -m venv venv
source venv/bin/activate
pip install locust requests
```

- Example for **REST**:

```
from locust import HttpUser, task

class ProductUser(HttpUser):
    @task
    def get_products(self):
        self.client.get("/products")
```

- Example for **GraphQL**:

```
from locust import HttpUser, task

class GraphQLUser(HttpUser):
    @task
    def get_products(self):
        query = '{"query": "{ products { uuid name weight } }"}'
        self.client.post("/graphql", data=query, headers={"Content-Type": "application/json"})
```

- Run:

```
locust -f locustfile.py
```

Resources